

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ

з лабораторної роботи №4

з дисципліни "Основи програмної інженерії"

Виконала

ст. гр. ПЗП-25-3

Головко Є.А.

Прийняли:

Каук В. І.

Гребенюк В. О.

Харків 2026

1. Тема та мета лабораторної роботи.

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2. Результати Code Review одногрупника:

№	Рядок коду	Проблема	Категорія	Рекомендація
1	22 (double result = (met * weight * 3.5 / 200)...)	Використання незрозумілих числових значень у базовій формулі розрахунку.	Magic Numbers	Винести числа 3.5 та 200 у константи класу з описовими іменами (наприклад, OXYGEN_CONSUMPTION_FACTOR).
2	31 (int maxHeartRate = 220 - age;)	Число 220 прямо в коді як незмінна база для розрахунку пульсу.	Magic Numbers	Винести число як константу на рівні класу: private const int BASE_MAX_HEART_RATE = 220;.
3	34 (if (intensity >= 0.85))	Пороги інтенсивності не мають імен, через що незрозуміло,	Magic Numbers	Створити іменовані константи (наприклад, ANAEROBIC_THRESHOLD = 0.85;).

		звідки беруться ці межі.		
4	15 ("bir" => 8.0,)	Використання звичайних рядків (string) для типу активності замість жорсткої типізації.	Primitive Obsession / Magic Strings	Створити перерахування enum ActivityType { Running, Swimming, Strength } і використовувати його.
5	53 (class Program)	Наявність порожнього класу та методу Main, які ніколи не виконуються в коді.	Dead Code	Змінити тип проєкту в .csproj на бібліотеку класів (<OutputType>Library</OutputType>) та видалити клас.
6	29 (if (age < 10 age > 100))	Межі допустимого віку 10 та 100 вбудовані прямо в логіку перевірки.	Magic Numbers	Винести межі в константи: MIN_ALLOWED_AGE = 10; та MAX_ALLOWED_AGE = 100;.
7	47 (double progress = (currentKcal / targetKcal) * 100;)	Число 100 дублюється тричі: як множник для відсотків і як	Magic Numbers / Duplicate Code	Винести значення у константу private const double MAX_PROGRESS_PERCENT = 100.0;,, щоб уникнути дублювання.

		максимальний ліміт.		
8	15-18 ("біг" => 8.0, "плавання" => 7.0...)	Показники MET прописані прямо всередині конструкції вибору switch.	Magic Numbers	Винести ці показники у словник (Dictionary) або окремі іменовані константи (напр., <code>RUNNING_MET = 8.0;</code>).

3. Посилання на PR з коментарями Code Review (URL).

<https://github.com/yelyzavetaholovko-ui/Impulse/pull/3>

4. Результати рефакторингу власного коду:

1) Операція 1: Усунення дублювання коду (Extract Variable)

БУЛО: `if (gender.ToLower() == "male" || gender.ToLower() == "чоловік")`

СТАЛО: `string normalizedGender = gender.ToLower(); if (normalizedGender == "male" || normalizedGender == "чоловік")`

ЧОМУ: Метод `ToLower()` викликався багаторазово, що порушує принцип DRY (Don't Repeat Yourself). Збереження результату в окрему змінну `normalizedGender` оптимізує код і робить його чистішим.

2) Операція 2: Заміна магічних чисел на константи (Replace Magic Number with Symbolic Constant)

БУЛО: `double extraWaterLiters = (workoutMinutes / 30.0) * 0.5;`

СТАЛО: `private const double WORKOUT_INTERVAL_MIN = 30.0; private const double WATER_PER_INTERVAL_LITERS = 0.5; double extraWaterLiters =`

(workoutMinutes / WORKOUT_INTERVAL_MIN) *
WATER_PER_INTERVAL_LITERS;

ЧОМУ: Числа 30.0 та 0.5 були "магічними" і не мали зрозумілого контексту. Винесення їх у константи робить формулу самодокументованою та зрозумілою без додаткових пояснювальних коментарів, а також дозволяє легко змінити ці норми в майбутньому в одному місці.

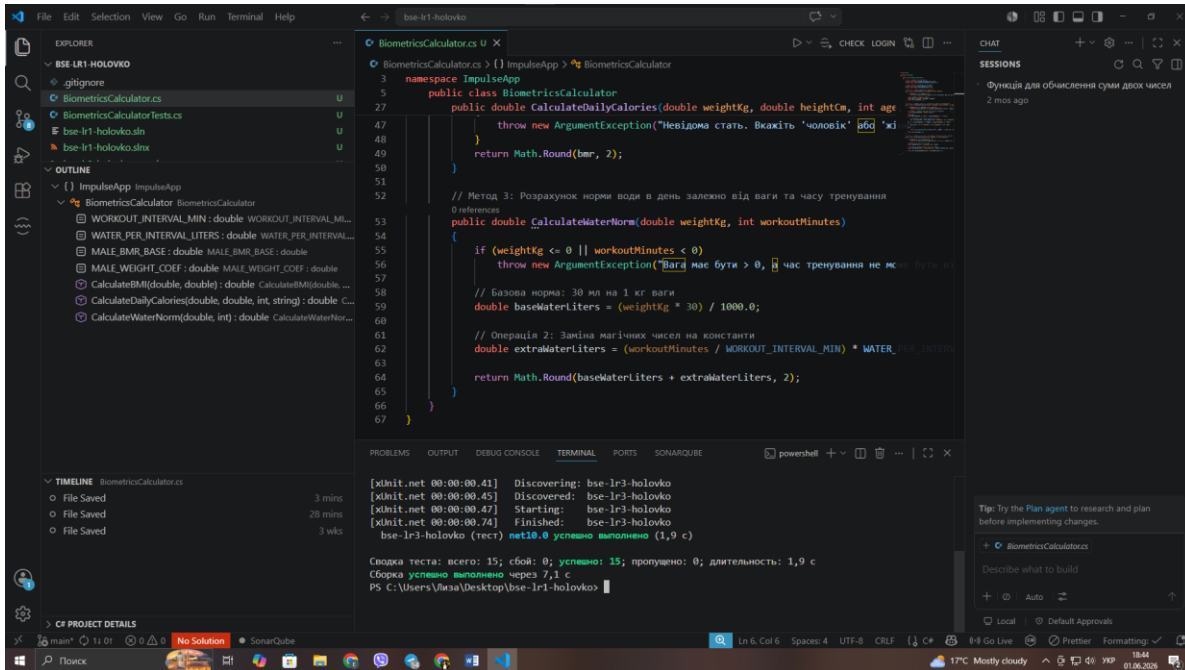
3) Операція 3: Рефакторинг математичних формул (Extract Constants)

БУЛО: $bmr = 88.36 + (13.4 * weightKg) + (4.8 * heightCm) - (5.7 * age);$

СТАЛО: private const double MALE_BMR_BASE = 88.36; private const double
MALE_WEIGHT_COEF = 13.4; bmr = MALE_BMR_BASE +
(MALE_WEIGHT_COEF * weightKg) + (4.8 * heightCm) - (5.7 * age);

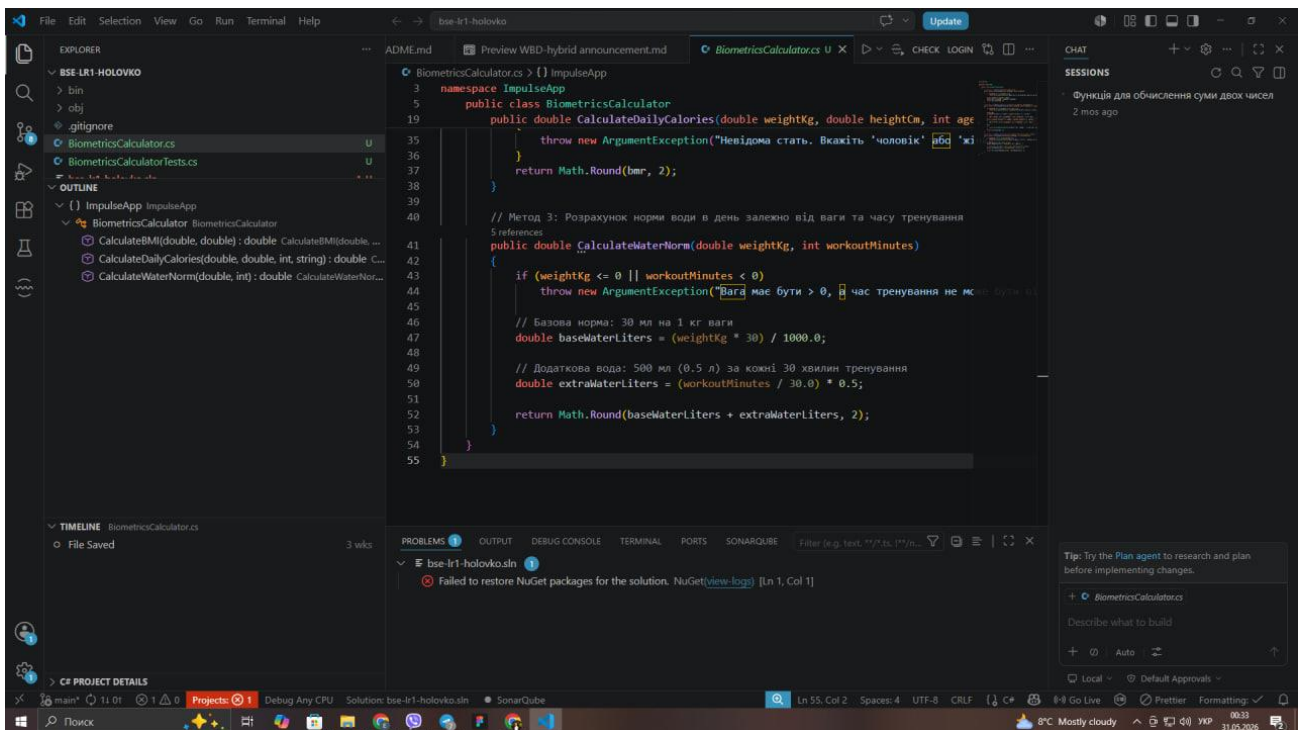
ЧОМУ: Використання "защитих" числових коефіцієнтів безпосередньо у формулі ускладнює читабельність. Створення іменованих констант дозволяє набагато легше та безпечніше звіряти базові коефіцієнти розрахунків зі стандартними методичними посібниками (формулою Харріса-Бенедикта), запобігаючи випадковим помилкам.

2) Звіт регресійного тестування:

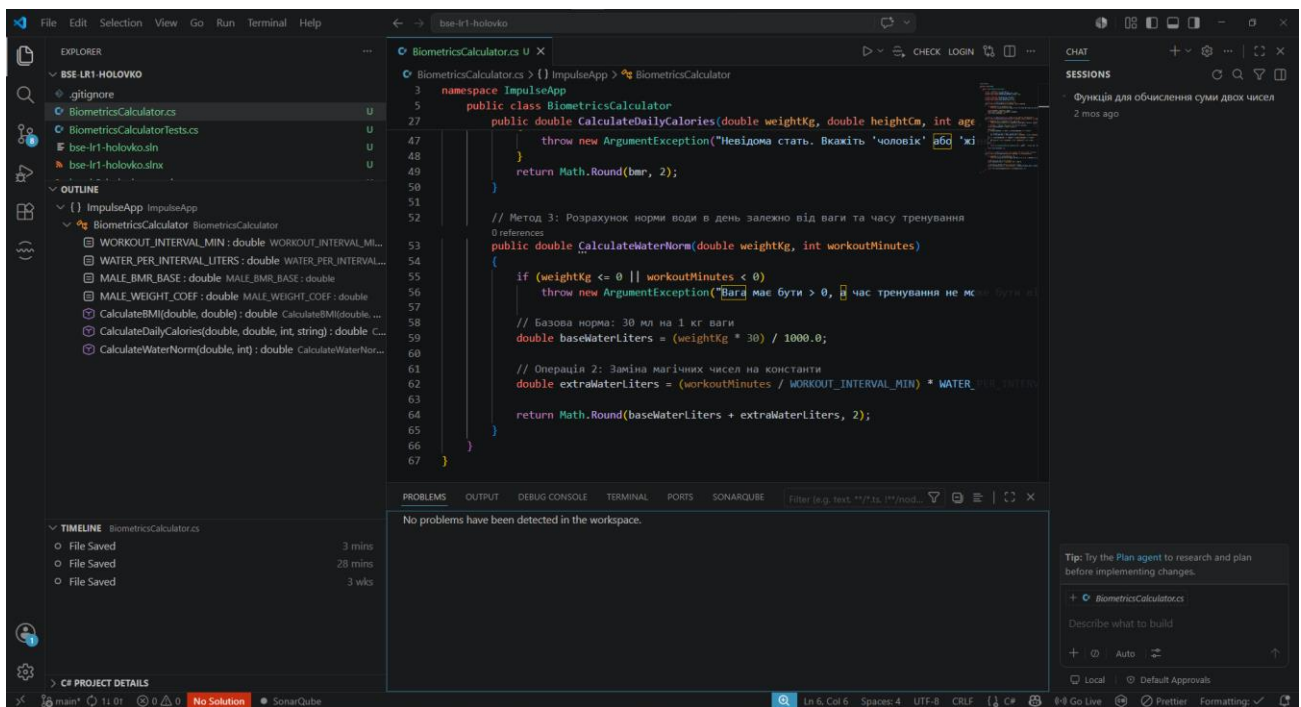


3) Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.

ДО:



ПІСЛЯ:



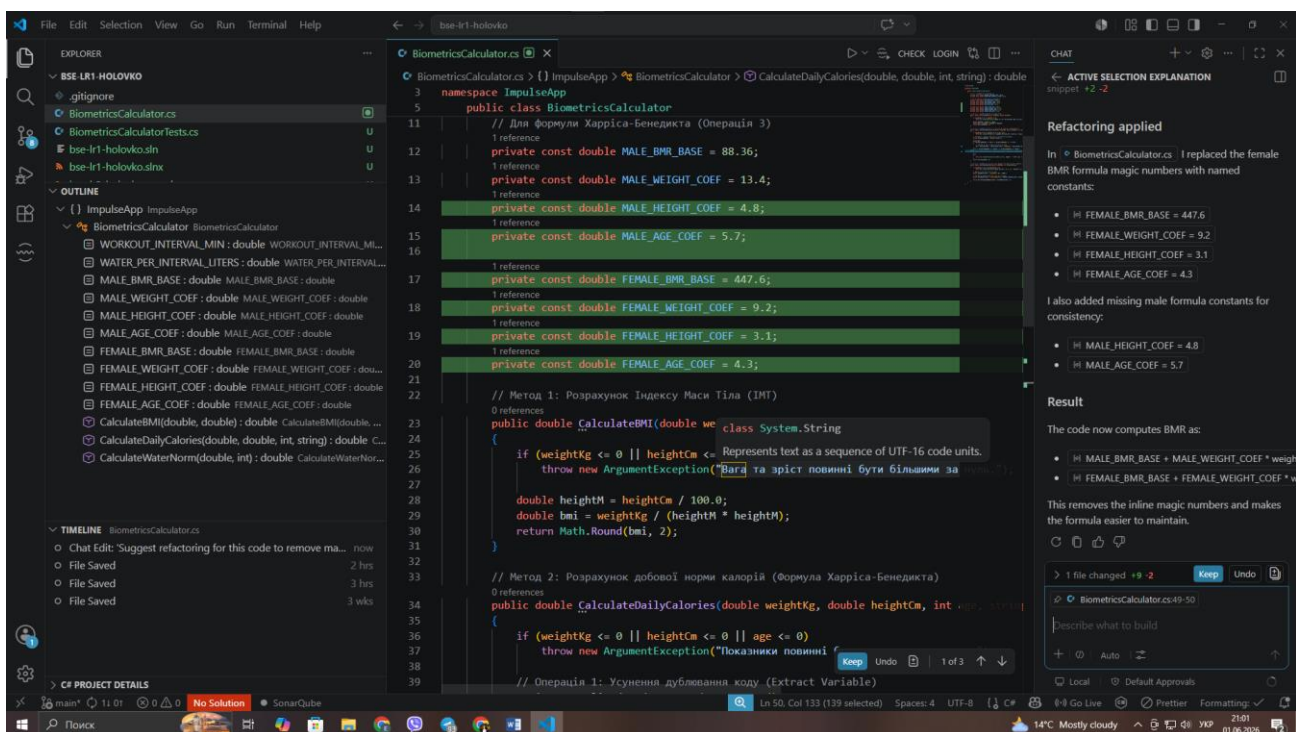
Цикломатична складність (Lizard):

ДО рефакторингу: 14 (загальна).

ПІСЛЯ рефакторингу: 14 (загальна).

Обґрунтування: Показник цикломатичної складності не змінився, оскільки проведені операції рефакторингу (Extract Variable, Replace Magic Number) стосувалися оптимізації даних та усунення дублювання, і не вплинули на кількість розгалужень чи логічних блоків у методах.

4) **Бонусне завдання:** пояснення GitHub Copilot Chat підозрілого фрагмента коду (Explain this code) або пропозиції рефакторингу(Suggest refactoring).



Асистенту було надіслано запит на рефакторинг фрагмента коду з формулою Харріса-Бенедикта для жінок, де залишилися магічні числа. Результат та пояснення від ШІ наведено на скріншоті.

5) Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review.

Найціннішим досвідом під час проведення Code Review стало усвідомлення, наскільки важливим є свіжий погляд іншого розробника для виявлення прихованих недоліків у структурі програми. Під час спільної перевірки коду з напарницею я на практиці засвоїла критичне значення усунення "магічних чисел" та уникнення дублювання логіки. Крім того, робота із Pull Requests на платформі GitHub навчила мене грамотно документувати знайдені проблеми та конструктивно пропонувати шляхи їх вирішення. Цей етап переконливо продемонстрував, що своєчасний рефакторинг не лише сприяє чистоті коду, а й значно спрощує його подальше обслуговування та модульне тестування.